KUBERNETES CHEAT SHEET + TOP 100 INTERVIEW QUESTIONS

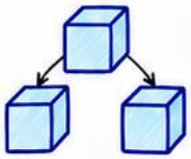
LEARN • PRACTICE • GET HIRED

1. PODS



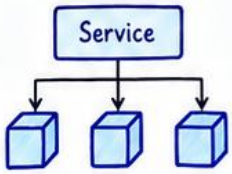
Smallest deployable unit
Runs one or more containers

2. DEPLOYMENTS



Manage stateless apps
Self-healing & scaling
(rollout/rollback)

3. SERVICES



Expose pods internally or externally
(ClusterIP, NodePort, LoadBalancer)

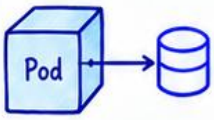
4. CONFIGMAP & SECRET


ConfigMap


Secret

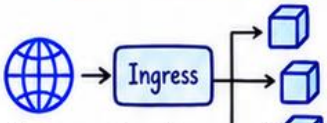
Store configuration and sensitive data as environment variables or files

5. VOLUMES & PERSISTENT STORAGE



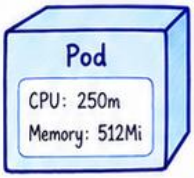
Persist data beyond pod lifecycle
(PVC, PV, StorageClass)

6. INGRESS & NETWORKING



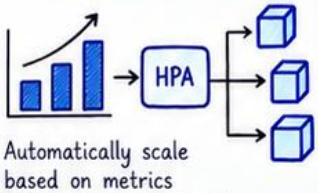
Manage external access and routing
(HTTP/HTTPS)

7. REQUESTS, LIMITS & PROBES



- Liveness Probe
- Readiness Probe
- Startup Probe

8. SCALING & HPA



Automatically scale based on metrics
(CPU/Memory/Custom)

9. STATEFULSETS, DAEMONSETS, JOBS, CRONJOBS


StatefulSet (Stateful)


DaemonSet (Run on all nodes)


Job (One-time)


CronJob (Scheduled)

10. SECURITY + RBAC + SERVICE ACCOUNTS




ServiceAccount

Control access and permissions
(Role, ClusterRole, RoleBinding)

KEY CONCEPTS

- ☑ Container Orchestration
- ☑ Declarative Configuration
- ☑ Self-healing
- ☑ Scalability
- ☑ High Availability
- ☑ Security & Access Control
- ☑ Monitoring & Troubleshooting



SAME CONCEPTS BIGGER OPPORTUNITIES!

11. MONITORING, LOGGING & TROUBLESHOOTING


Prometheus (Metrics)


Grafana (Visualization)


Logs (EFK / Loki)


Debug (kubectl)

12. INTERVIEW PREP



- ☑ Top 100 Questions with answers
- ☑ Cover concepts, scenarios and real-world use cases


PRACTICE TODAY
DEPLOY TOMORROW!


BETTER CLUSTERS
BRIGHTER FUTURE!

USEFUL COMMANDS



- kubectl get pods
- kubectl describe pod <name>
- kubectl logs <pod>
- kubectl apply -f <file.yaml>
- kubectl get all

COMMON YAML TEMPLATES



- Pod
- Deployment
- Service
- ConfigMap
- Secret
- PersistentVolumeClaim
- Ingress
- HPA

INTERVIEW CATEGORIES



- Basics & Concepts
- Workloads & Scheduling
- Networking & Services
- Storage & Volumes
- Security & RBAC
- Troubleshooting
- Real-world Scenarios

GOAL



- ☑ Learn
- ☑ Practice
- ☐ Crack Interviews
- ☐ Build Real Skills
- ☐ Grow in DevOps

You Got This!

1. Kubernetes Basics & Architecture

What is Kubernetes ?

Kubernetes (K8s) is an open-source container orchestration platform developed by Google. It helps you deploy, manage, scale and maintain containerized applications across a cluster of machines.

> **Note :**
> Kubernetes manages containers, not the containers themselves !

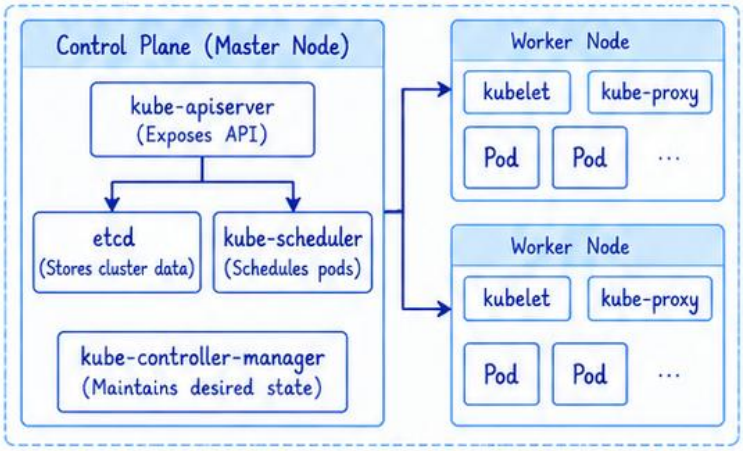
Why Kubernetes ?

- Automates deployment and scaling
- Self-healing (restart failed containers)
- Load balancing and service discovery
- Efficient resource utilization
- Runs on any infrastructure (cloud / on-prem)
- Large community and ecosystem
- Used in production by companies like Google, Netflix, Amazon, Microsoft, etc.

Key Features

- Container orchestration
- Automatic scaling
- Self-healing
- Service discovery & load balancing
- Rolling updates and rollbacks
- Works across multiple environments
- Strong community support

High-Level Architecture



Core Components

Pod	→	Smallest deployable unit
Deployment	→	Manages replica sets of pods
Service	→	Exposes pods and enables discovery
Ingress	→	Manages external access (HTTP/S)
ConfigMap	→	Stores configuration data
Secret	→	Stores sensitive data
Namespace	→	Logical isolation of resources
Node	→	Worker machine (runs pods)
Cluster	→	Set of nodes managed by Kubernetes

Simple Kubernetes Cluster Setup (Local)

```
# Install Minikube (for local Kubernetes)
brew install minikube    # Mac (or follow official docs)

# Start a local cluster
minikube start

# Check cluster status
kubectl cluster-info

# Get nodes
kubectl get nodes
```

kubectl – Basic Commands

```
kubectl get nodes              # List nodes
kubectl get pods               # List pods
kubectl get deployments        # List deployments
kubectl get services           # List services
kubectl describe pod <name>    # Get details
kubectl delete pod <name>      # Delete a pod
```

Containers Today
Better Systems Tomorrow !

From Concepts to Real Systems !

2. Pods + Containers

Smallest Unit Bigger Possibilities !

What is a Container ?

A container is a lightweight, standalone, executable package that includes everything an application needs to run – code, runtime, libraries, and dependencies. Containers share the host OS kernel and are isolated from each other.

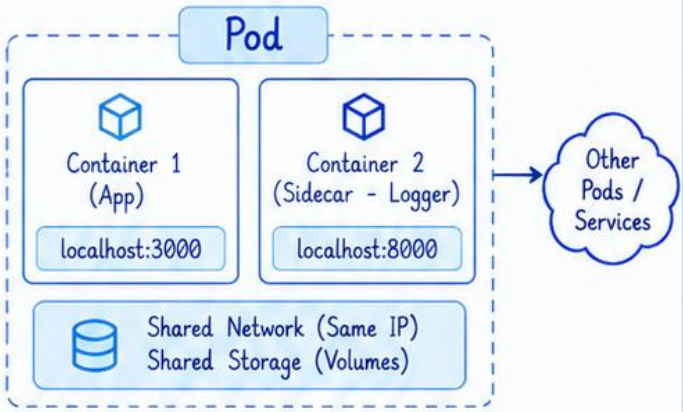
What is a Pod ?

A Pod is the smallest deployable unit in Kubernetes. It can contain one or more containers that share the same network namespace (same IP address) and storage. containers inside a pod can communicate with each other using localhost.

Docker Container vs Kubernetes Pod

Feature	Container (Docker)	Pod (Kubernetes)
Definition	Single application unit	Smallest deployable unit (can have one or more containers)
Managed by	Docker	Kubernetes
Networking	Own network	Shared network (same IP)
Storage	Isolated	Shared volumes
Use case	Run an application	Run and manage one or more containers together

Pod with Multiple Containers



Pod Characteristics

- Has a unique IP address
- Can have one or more containers
- Containers in a pod share network and storage
- Always scheduled on the same node
- Ephemeral by nature (can be recreated)
- Managed by higher-level controllers (Deployments, ReplicaSets)
- Typically runs a single application per pod (multi-container is for sidecars like logging, monitoring, etc.)

Simple Pod Example (YAML)

```yaml
apiVersion: v1                  # Kubernetes API version
kind: Pod                       # Resource type
metadata:
  name: my-app-pod              # Pod name
spec:
  containers:                   # Define containers
    - name: app-container       # Container name
      image: nginx:latest       # Container image
      ports:
        - containerPort: 80     # Expose port
```

Multiple Containers in a Pod (YAML)

```yaml
apiVersion: v1                  # Pod with two containers
kind: Pod
metadata:
  name: app-with-logger
spec:
  containers:
    - name: app                 # Main application container
      image: myapp:latest
    - name: logger
      image: fluentd:latest     # Sidecar container for logging
```

Common Use Cases

- Sidecar containers (logging, monitoring, proxy)
- Tightly coupled applications (e.g., app + helper)
- Sharing files via shared volumes
- Shared network (localhost communication)
- Init containers (setup before main container)

Build Deploy Scale Repeat !

Same Containers Different Possibilities !

From Concepts to Real Systems !

3. Deployments + ReplicaSets

Manage Scale Heal Update ! ☺

@codedsoul_05

What is a ReplicaSet ?

A ReplicaSet ensures that a specified number of pod replicas are always running. It continuously monitors the running pods and creates or deletes pods to maintain the desired count.

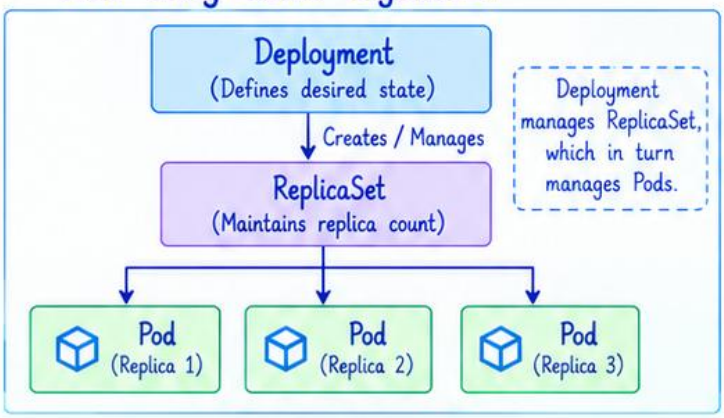
Guarantees high availability by self-healing.

What is a Deployment ?

A Deployment is a higher-level controller that manages ReplicaSets and provides declarative updates for your application. It allows you to perform rolling updates, rollbacks, and manage the application lifecycle easily.

Use Deployments (not ReplicaSets) in real projects.

How They Work Together ?



Key Differences

Feature	Deployment	ReplicaSet
Purpose	Manages application lifecycle	Maintains a stable set of pod replicas
Level	Higher-level controller	Lower-level controller
Manages	Creates and manages ReplicaSets	Directly manages Pods
Updates	Supports rolling updates and rollbacks	No built-in update mechanism
Use Case	Recommended for applications	Used by Deployment (rarely used directly)

Deployment YAML Example

```yaml
apiVersion: apps/v1              # Deployment API version
kind: Deployment                 # Resource type
metadata:
  name: my-app                   # Deployment name
spec:
  replicas: 3                    # Desired number of pods
  selector:
    matchLabels:
      app: my-app                # Select pods with this label
  template:
    metadata:
      labels:
        app: my-app              # Pod label
    spec:
      containers:
      - name: app
        image: nginx:latest      # Container image
        ports:
        - containerPort: 80      # Expose port
```

ReplicaSet YAML Example

```yaml
apiVersion: apps/v1              # ReplicaSet API version
kind: ReplicaSet                 # Resource type
metadata:
  name: my-app-rs                # ReplicaSet name
spec:
  replicas: 3                    # Desired number of pods
  selector:
    matchLabels:
      app: my-app                # Select pods with this label
  template:
    metadata:
      labels:
        app: my-app              # Pod label
    spec:
      containers:
      - name: app
        image: nginx:latest      # Container image
        ports:
        - containerPort: 80      # Expose port
```

Common Commands

```
kubectl get deployments                          # List deployments
kubectl get replicasets                          # List replicasets
kubectl describe deployment <name>               # Get details
kubectl scale deployment <name> --replicas=5     # Scale pods
kubectl rollout status deployment <name>         # Check rollout status
kubectl rollout undo deployment <name>           # Rollback to previous version
```

Use Cases

- Deploy and manage stateless applications
- Rolling updates with zero downtime
- Automatic self-healing (replaces failed pods)
- Easy horizontal scaling
- Rollback to previous versions
- Used in production for high availability

Reliable Applications Happy Users ! ☺

@codedsoul_05

Deploy Scale Improve Repeat ! ☺

From Concepts to Real Systems!

4. Services + Service Discovery

@codedsoul_05

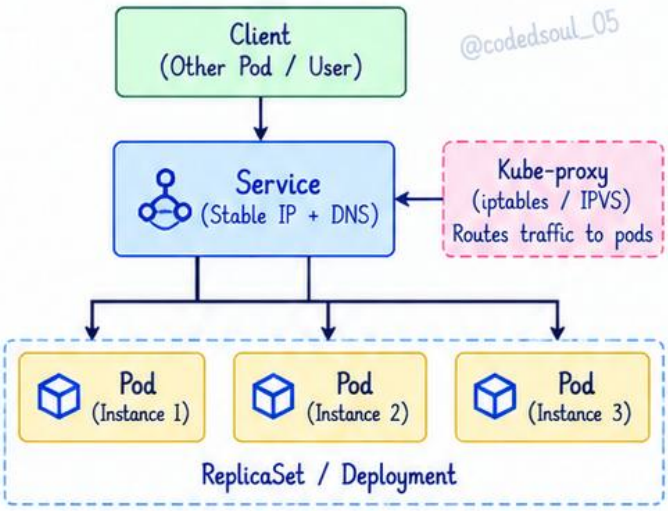
Connect Discover Scale Seamlessly!

What is a Service ?

A Service in Kubernetes is an abstraction that exposes a set of Pods as a network service. It provides a stable IP address and DNS name, and load balances traffic to the healthy pods behind it.

> Services decouple your application from the underlying pods.

How Service Works ?



Service YAML Example (ClusterIP)

```yaml
apiVersion: v1              # Kubernetes API version
kind: Service               # Resource type
metadata:
  name: my-service          # Service name
spec:
  type: ClusterIP           # Service type
  selector:
    app: my-app             # Select pods with this label
  ports:
    - protocol: TCP
      port: 80              # Service port
      targetPort: 8080      # Pod container port
```

Useful Commands

```
kubectl get services                  # List all services
kubectl describe service <name>       # Get service details
kubectl get endpoints                 # See service endpoints
kubectl get pods -l app=my-app        # Check selected pods
```

Why Do We Need a Service ?

- Pods are ephemeral (can be created, deleted, replaced)
- Each pod has a different IP (changes over time)
- Services provide a stable endpoint (IP + DNS)
- Automatic load balancing across healthy pods
- Enables service discovery within the cluster
- Supports different access types (ClusterIP, NodePort, etc.)

Types of Services

Type	Description	Use Case
ClusterIP	Default type. Exposes service inside the cluster with a virtual IP.	Internal communication between microservices
NodePort	Exposes service on a static port on each node (NodeIP:Port).	Access from outside the cluster (dev/testing)
LoadBalancer	Creates an external load balancer (in cloud) and exposes the service.	Production (cloud environments)
ExternalName	Maps service to an external DNS name (no proxying).	Access external services (e.g., database)

Service Discovery in Kubernetes

- Each service gets a DNS name in the format:
  `<service-name>.<namespace>.svc.cluster.local`
- Kubernetes uses CoreDNS for DNS-based service discovery.
- Pods can use the service name instead of IP address.
- Works across namespaces using: `<service>.<namespace>.svc.cluster.local`

Accessing a Service

- From another pod (same namespace):
  `http://my-service:80`
- From another namespace:
  `http://my-service.<namespace>.svc.cluster.local`
- From outside (NodePort):
  `http://<NodeIP>:<NodePort>`
- From outside (LoadBalancer):
  `http://<External-IP>:80`

@codedsoul_05

Key Takeaways

- ✓ Services provide a stable endpoint for pods
- ✓ Enable load balancing and service discovery
- ✓ Multiple types for different use cases
- ✓ Use DNS names instead of IPs
- ✓ Essential for building scalable microservices

Build Better Systems One Service at a Time!

Stable Endpoints Happier Deployments!

From Concepts to Real Systems!

5. Namespaces + Labels + Selectors

Organize Identify Select Scale !

@codedsoul_05

What is a Namespace ?

A Namespace is a logical isolation unit in Kubernetes that allows you to divide cluster resources between multiple teams, environments, or applications. Resources in one namespace are isolated from others (by default).

Same cluster, multiple isolated environments !

Why Use Namespaces ?

- Logical isolation of resources
- Separate environments (dev, staging, prod)
- Access control and resource quotas
- Avoid name conflicts
- Manage multi-tenant clusters

Namespace Example (YAML)

```yaml
apiVersion: v1          # Kubernetes API version
kind: Namespace         # Resource type
metadata:
  name: dev             # Namespace name
```

What are Labels ?

Labels are key-value pairs attached to Kubernetes resources (e.g., pods, deployments, services) to organize and identify them. Labels do not provide uniqueness, and a resource can have multiple labels.

Labels are for identification and grouping.

What are Selectors ?

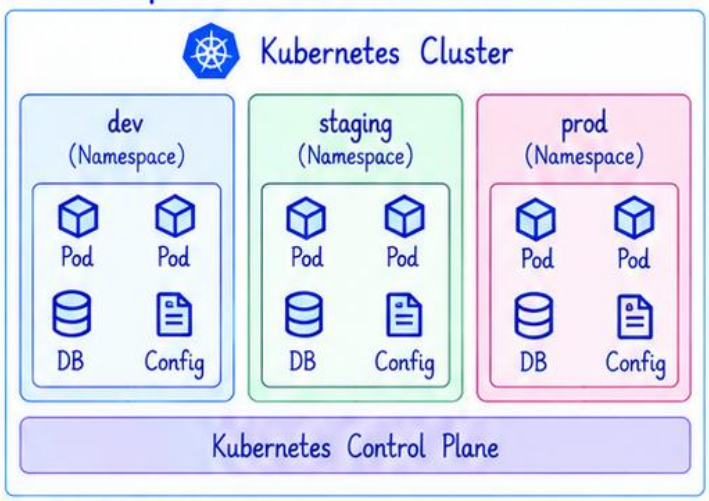
Selectors are used to filter and select resources based on labels. They are used by higher-level objects like Deployments, ReplicaSets, and Services to find and manage the right set of pods.

Selectors match labels to group resources.

Label Selectors in Different Objects

Object	Uses Selectors ?	Purpose
Deployment	Yes	Selects pods to manage
ReplicaSet	Yes	Selects pods to maintain count
Service	Yes	Selects pods to route traffic
DaemonSet	Yes	(Uses node selectors, not labels)
StatefulSet	Yes	Selects pods to manage
Job / CronJob	No	Creates pods directly (no selector)

Namespaces in Action



Useful Commands

```
kubectl get namespaces              # List all namespaces
kubectl create namespace <name>     # Create a namespace
kubectl get pods -n <name>          # List pods in a namespace
kubectl delete namespace <name>     # Delete a namespace
kubectl config set-context --current --namespace=<name> # Switch namespace
```

Label Example (YAML)

```yaml
apiVersion: v1          # Pod with labels
kind: Pod
metadata:
  name: my-app
  labels:
    app: my-app         # key: value
    tier: backend
    env: prod
```

A resource can have multiple labels!

Selector Example (YAML)

```yaml
apiVersion: apps/v1     # Deployment using label selector
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app       # Select pods with this label
  template:
    metadata:
      labels:
        app: my-app     # Pods will have this label
    spec:
      containers:
      - name: app
        image: nginx:latest
```

Selector matches pods with app: my-app

Organize Today For a Scalable Tomorrow!

Right Labels Right Resources Bigger Possibilities!

From Concepts to Real Systems !

6. ConfigMaps + Secrets

@codedsoul_05

Keep Configuration Flexible and Secure ! ☺

What is a ConfigMap ?

A ConfigMap is used to store non-sensitive configuration data in key-value format. It allows you to externalize application configuration from your container image.

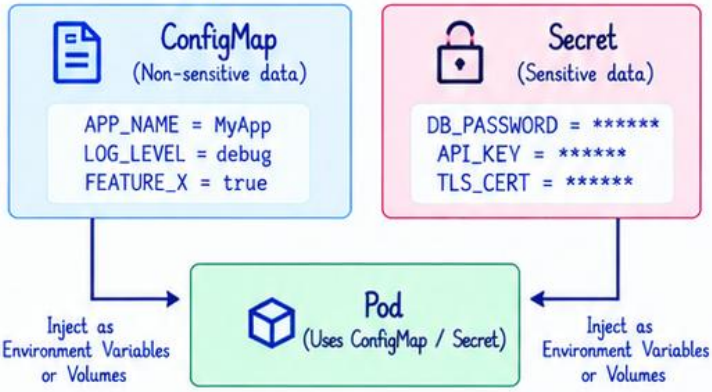
Keep your configuration separate from your code !

What is a Secret ?

A Secret is used to store sensitive information like passwords, API keys, tokens, and certificates in an encoded format (base64). It helps keep sensitive data out of your container images and source code.

Use Secrets for sensitive data, not ConfigMaps !

How They Work ?



ConfigMap Example (YAML)

```yaml
apiVersion: v1          # Kubernetes API version
kind: ConfigMap         # Resource type
metadata:
  name: app-config      # ConfigMap name
data:
  APP_NAME: "MyApp"     # Key-value pairs
  LOG_LEVEL: "debug"
  FEATURE_X: "true"
```

Store non-sensitive configuration data !

Secret Example (YAML)

```yaml
apiVersion: v1          # Kubernetes API version
kind: Secret            # Resource type
metadata:
  name: db-secret       # Secret name
type: Opaque            # Type (Opaque, kubernetes.io/tls, etc.)
data:
  DB_PASSWORD: cGFzc3dvcmQxMjM=   # base64 encoded value
  API_KEY: YWbjMIIzND52           # base64 encoded value
```

Values are base64 encoded !

Using ConfigMap in a Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
    - name: app
      image: nginx:latest
      env:
      - name: APP_NAME
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: APP_NAME
```

Use config data as environment variables !

Using Secret in a Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
spec:
  containers:
    - name: app
      image: nginx:latest
      env:
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: DB_PASSWORD
```

Use sensitive data as environment variables !

ConfigMap vs Secret

Feature	ConfigMap	Secret
Data Type	Non-sensitive	Sensitive
Encoded	Plain text	Base64 encoded
Use Case	Configuration (e.g. app settings)	Passwords, API keys, tokens
Size Limit	1 MiB	1 MiB
Type	N/A	Opaque, TLS, Docker config, etc.
Security	Not encrypted (by default)	Can be encrypted (at rest)

Useful Commands

```
kubectl get configmaps                # List all ConfigMaps
kubectl describe configmap <name>     # Get details
kubectl apply -f configmap.yaml       # Create/Update
kubectl get secrets                   # List all Secrets
kubectl describe secret <name>        # Get details
kubectl apply -f secret.yaml          # Create/Update
```

Separate Config Secure Secrets Build Better ! ☺

@codedsoul_05

Right Config Right Security Brighter Deployments ! ☺

From Concepts to Real Systems!

7. Volumes + Persistent Storage

Store Persist Scale Your Data!

@codedsoul_05

What is a Volume ?

A Volume provides a way to persist data beyond the lifecycle of a container. Containers are ephemeral (temporary), so data inside a container is lost when the container is deleted. Volumes allow you to store data on the node or on external storage.

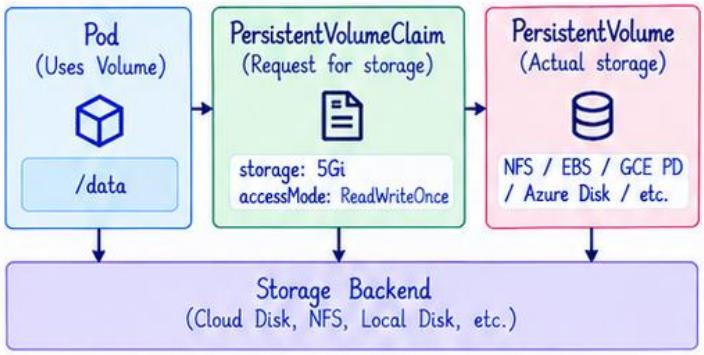
> Volumes survive container restarts, but are tied to the pod's lifecycle.

What is Persistent Storage ?

Persistent Storage in Kubernetes is handled using PersistentVolumes (PV) and PersistentVolumeClaims (PVC). It allows pods to use storage that exists independently of the pod's lifecycle, so data is not lost even if the pod is deleted or rescheduled.

> PV is the storage resource in the cluster, PVC is a request for storage by a user.

How it Works ?



Types of Volumes

Type	Description	Use Case
emptyDir	Temporary storage, deleted when pod is removed	Caching, temporary files
hostPath	Mounts a file/directory from the node	Node-level access (dev/testing)
nfs	Network File System	Shared storage across pods
cloud storage (e.g., EBS, GCE PD)	Persistent disk from cloud provider	Production workloads
configMap / secret	Mounts config or secret as files	Configuration files
persistentVolume (PV)	Cluster-wide persistent storage	Long-term storage
persistentVolumeClaim (PVC)	Request for storage	Used by pods to claim PV

emptyDir Volume Example (YAML)

```yaml
apiVersion: v1                    # Pod with emptyDir volume
kind: Pod
metadata:
  name: my-app
spec:
  containers:
   - name: app
     image: nginx:latest
     volumeMounts:
      - name: cache               # Volume name
        mountPath: /data          # Mount path inside container
  volumes:
    - name: cache
      emptyDir: {}                # Define volume
                                  # Temporary storage
```

Data is lost when the pod is deleted !

PersistentVolume & PersistentVolumeClaim Example (YAML)

```yaml
apiVersion: v1                    # PersistentVolume definition
kind: PersistentVolume
metadata:
  name: my-pv                     # PV name
spec:
  capacity:
    storage: 10Gi                 # Storage size
  accessModes:
   - ReadWriteOnce                # Access mode
  hostPath:
    path: /mnt/data               # Storage location
```

PV (Cluster Resource)

```yaml
apiVersion: v1                    # PersistentVolumeClaim definition
kind: PersistentVolumeClaim
metadata:
  name: my-pvc                    # PVC name
spec:
  accessModes:
   - ReadWriteOnce
  resources:                      # Access mode
    requests:
      storage: 10Gi               # Requested storage size
```

PVC (User Request)

Common Access Modes

Access Mode	Description
ReadWriteOnce (RWO)	Can be mounted as read-write by a single node
ReadOnlyMany (ROX)	Can be mounted as read-only by multiple nodes
ReadWriteMany (RWX)	Can be mounted as read-write by multiple nodes
ReadWriteOncePod (RWOP)	Can be mounted as read-write by a single pod (Kubernetes 1.22+)

Useful Commands

```
kubectl get pv                    # List all PersistentVolumes
kubectl get pvc                   # List all PersistentVolumeClaims
kubectl describe pv <name>        # Get PV details
kubectl describe pvc <name>       # Get PVC details
```

Key Takeaways

- ✓ Use volumes to persist data beyond container lifecycle
- ✓ Use PV & PVC for long-term persistent storage
- ✓ Choose the right access mode for your use case
- ✓ Supports multiple storage backends (cloud, NFS, local)
- ✓ Essential for stateful applications (databases, file storage, etc.)

Persistent Data Happier Applications!

@codedsoul_05

Right Storage Right Scale Brighter Future!

From Concepts to Real Systems !

8. Ingress + Networking

@codedsoul_05

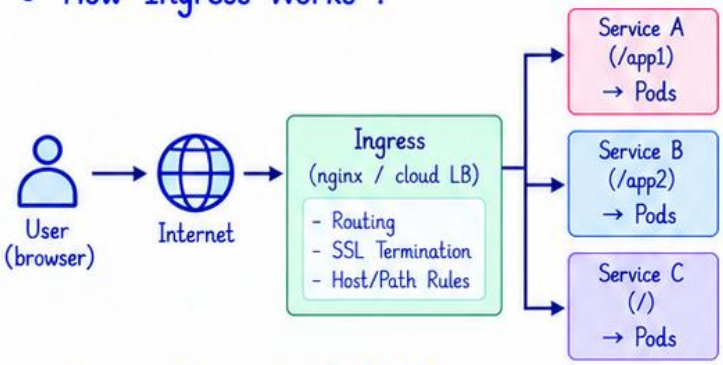
Expose Connect Secure Scale !

What is Ingress ?

Ingress is an API object that manages external access to services in a Kubernetes cluster, typically HTTP/HTTPS. It provides features like hosting multiple services under a single IP, path-based routing, SSL/TLS termination, and more.

Ingress acts as a reverse proxy for your services.

How Ingress Works ?



Ingress Example (YAML)

```yaml
apiVersion: networking.k8s.io/v1     # Ingress API version
kind: Ingress                        # Resource type
metadata:
  name: my-ingress                   # Ingress name
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    kubernetes.io/ingress.class: nginx     # Ingress controller
spec:
  rules:
  - host: example.com                # Domain name
    http:
      paths:
      - path: /app1                  # Path for service A
        pathType: Prefix
        backend:
          service:
            name: app1-service       # Service name
            port:
              number: 80             # Service port
      - path: /app2                  # Path for service B
        pathType: Prefix
        backend:
          service:
            name: app2-service
            port:
              number: 80
```

Routes traffic based on host and path !

Useful Commands

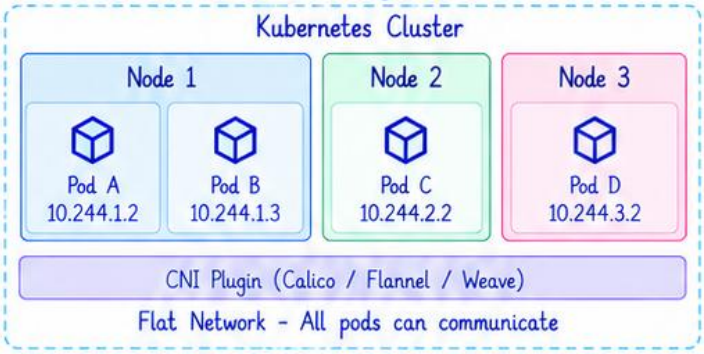
```
kubectl get ingress                      # List all ingresses
kubectl describe ingress <name>          # Get details
kubectl get ingress -n <namespace>       # In a namespace
kubectl get svc                          # List services
kubectl get endpoints                    # Check service endpoints
```

What is Networking in Kubernetes ?

Kubernetes provides a flat, unified network where every pod can communicate with every other pod using IP addresses. Networking is handled by a Container Network Interface (CNI) plugin (like Calico, Flannel, Weave, etc.).

Every pod gets a unique IP and can talk to any other pod without NAT.

Kubernetes Networking Model



Types of Kubernetes Networking

Type	Description	Use Case
Pod Networking	Each pod has a unique IP, can communicate with any pod in the cluster.	Pod-to-pod communication
Service Networking	Stable virtual IP (ClusterIP, NodePort, LoadBalancer)	Access to pods via services
Ingress Networking	Manages external HTTP/HTTPS access to services.	Expose web applications
Network Policies	Control traffic between pods using rules.	Security and access control
CNI Plugins	Implement the network layer (e.g., Calico, Flannel, Weave).	Enable pod networking

Service vs Ingress

Feature	Service	Ingress
Protocol	TCP/UDP	HTTP/HTTPS
Use Case	Internal access	External web access
Routing	Port-based	Host/Path-based
SSL/TLS	No (handled by LB)	Yes (termination)
Resource Type	v1/Service	networking.k8s.io/v1/Ingress

Key Takeaways

- ✓ Ingress provides external access to services
- ✓ Use path-based or host-based routing
- ✓ Kubernetes has a flat network (pod-to-pod communication)
- ✓ CNI plugin enables the networking layer
- ✓ Use Network Policies for security
- ✓ Combine Ingress + TLS for secure applications

Open More Possibilities With Kubernetes !

@codedsoul_05

Connect Route Protect Grow !

From Concepts to Real Systems!

9. Requests + Limits + Probes

Healthy Pods Happy Applications!

@codedsoul_05

What are Requests and Limits ?

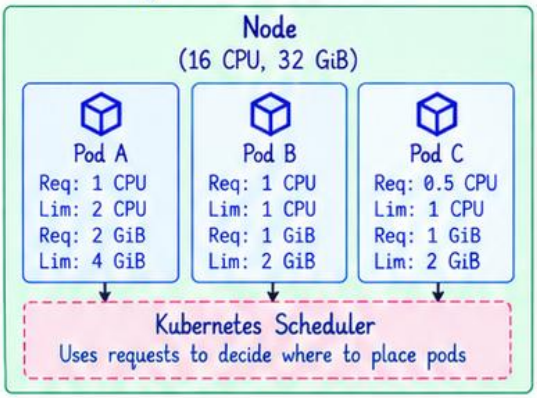
Requests and Limits define how much CPU and Memory a container can use. They help Kubernetes in scheduling pods and ensuring fair resource allocation across the cluster.

Requests = Minimum guaranteed resources
Limits = Maximum allowed resources

Why are They Important ?

- Helps in efficient resource allocation
- Prevents a single pod from consuming all resources
- Improves application stability and performance
- Enables better cluster scheduling
- Avoids OutOfMemory (OOM) and CPU throttling issues

How Requests and Limits Work ?



Requests are used for scheduling, Limits are enforced at runtime !

YAML Example (Requests & Limits)

```yaml
apiVersion: v1      # Pod with resource requests and limits
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
  - name: app
    image: nginx:latest
    resources:
      requests:
        cpu: "500m"          # 0.5 CPU
        memory: "256Mi"      # 256 MiB
      limits:
        cpu: "1"             # 1 CPU
        memory: "512Mi"      # 512 MiB
```

Units :
CPU: 1 = 1 core
500m = 0.5 core
Memory: 1Gi = 1 GiB
512Mi = 512 MiB

Units for Resources

Resource	Unit	Example	Meaning
CPU	m (millicore)	500m	0.5 CPU
	(core)	1	1 CPU
Memory	Mi (Mebibyte)	256Mi	256 MiB
	Gi (Gibibyte)	1Gi	1 GiB

What are Probes ?

Probes are health checks used by Kubernetes to determine the state of a container. They help in automatically detecting and recovering from failures.

Probes ensure your application is Running, Ready and Alive !

Types of Probes

Probe Type	Description	Use Case
Liveness Probe	Checks if the container is running. If it fails, Kubernetes restarts the container.	Detect deadlocks or hung applications
Readiness Probe	Checks if the container is ready to serve traffic. If it fails, traffic is not sent to the pod.	Wait for app to be fully ready (e.g. DB connection)
Startup Probe	Checks if the application has started. It gives initial startup time before liveness kicks in.	For applications that take longer to start

YAML Example (Probes)

```yaml
apiVersion: v1          # Pod with probes
kind: Pod
metadata:
  name: probe-demo
spec:
  containers:
  - name: app
    image: nginx:latest
    ports:
    - containerPort: 80
    livenessProbe:
      httpGet:
        path: /health
        port: 80
        initialDelaySeconds: 10
        periodSeconds: 5
    readinessProbe:
      httpGet:
        path: /ready
        port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
    startupProbe:
      httpGet:
        path: /start
        port: 80
        failureThreshold: 30
        periodSeconds: 10
```

Liveness Probe
Restarts container if it fails.

Readiness Probe
Controls traffic to the pod.

Startup Probe
Gives time for application to start before liveness kicks in.

Key Takeaways

- ✓ Set realistic CPU and memory requests & limits
- ✓ Use probes to make your application self-healing
- ✓ Use readiness probe to avoid sending traffic too early
- ✓ Use startup probe for slow-starting applications
- ✓ Proper resource configuration improves stability and efficiency

Right Resources Stronger Applications!

@codedsoul_05

Configure Today Scale Tomorrow!

From Concepts to Real Systems !

10. Configuring Applications + Environment Variables

Configure Adapt Deploy Scale ! ☺

@codedsoul_05

What does it mean ?

Configuring applications in Kubernestes means providing configuration data (like app settings, feature flags, API URLs, etc.) to your containers without changing the container image. Environment variables are a simple way to pass this configuration to your application at runtime.

Same image, different configuration for dev, staging and prod !

Why use External Configuration ?

- Keep configuration separate from code
- Easily manage different environments (dev, staging, prod)
- Update configuration without rebuilding the image
- Use ConfigMaps for non-sensitive data
- Use Secrets for sensitive data (passwords, API keys, etc.)
- Makes applications more flexible and scalable

Ways to Configure Applications

Method	Use For	Example
Environment Variables	Simple key-value configuration	APP_ENV=prod
ConfigMap	Non-sensitive configuration files or values	app.properties, config.yaml
Secret	Sensitive data (passwords, tokens, keys)	DB_PASSWORD, API_KEY
Volume Mount	Mount configuration files into containers	/etc/config/app.yaml

ConfigMap Example (Environment Variables)

```yaml
apiVersion: v1              # ConfigMap definition
kind: ConfigMap
metadata:
  name: app-config
  namespace: default
data:
  APP_ENV: "production"
  API_URL: "https://api.example.com"
  FEATURE_X: "true"
```

Store non-sensitive configuration data !

Secret Example (Environment Variables)

```yaml
apiVersion: v1              # Secret definition
kind: Secret
metadata:
  name: app-secret
  namespace: default
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQxMjM=     # base64 encoded
  API_KEY: dG9rZW4xMjM=             # base64 encoded
```

Store sensitive data securely !

Using ConfigMap and Secret in a Pod

```yaml
apiVersion: v1              # Pod definition
kind: Pod
metadata:
  name: my-app
spec:
  containers:
  - name: app
    image: my-app:latest
    env:
      - name: APP_ENV
        valueFrom:
          configMapKeyRef:
            name: app-config      # ConfigMap name
            key: APP_ENV          # Key in ConfigMap
      - name: API_URL
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: API_URL
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: app-secret      # Secret name
            key: DB_PASSWORD      # Key in Secret
      - name: API_KEY
        valueFrom:
          secretKeyRef:
            name: app-secret
            key: API_KEY
```

Use ConfigMap for app settings and Secret for sensitive data !

Accessing Environment Variables in Code

Python

```python
import os
app_env = os.getenv("APP_ENV")
api_url = os.getenv("API_URL")
print(f"Environment: {app_env}")
```

Node.js

```javascript
const appEnv = process.env.APP_ENV;
const apiUrl = process.env.API_URL;
console.log(`Environment: ${appEnv}`);
```

Java

```java
String appEnv = System.getenv("APP_ENV");
String apiUrl = System.getenv("API_URL");
System.out.println("Environment: " + appEnv);
```

Useful Commands

```
kubectl create configmap <name> --from-literal=KEY=value    # Create ConfigMap
kubectl create secret generic <name> --from-literal=KEY=value # Create Secret
kubectl get configmaps                                      # List ConfigMaps
kubectl get secrets                                         # List Secrets
kubectl describe configmap <name>                           # Get details
kubectl describe secret <name>                              # Get details
```

Key Takeaways

- ✓ Use environment variables for flexible configuration
- ✓ Keep sensitive data in Secrets
- ✓ Use ConfigMaps for non-sensitive settings
- ✓ Separate configuration from application code
- ✓ Easily manage different environments
- ✓ Update configuration without rebuilding container images

Same Image Different Environments Infinite Possibilities ! ☺

@codedsoul_05

Configure Smart Build Better ! ☺

From Concepts to Real Systems !

11. Scaling + Autoscaling (HPA)

@codedsoul_05

Scale Smarter Not Harder !

Why Scaling is Important ?

Scaling helps your application handle more traffic, maintain performance, and ensure high availability. As demand increases (or decreases), you can adjust the number of pod replicas to match the load.

Right number of pods. Right performance. Always available !

Types of Scaling in Kubernetes

Manual Scaling

You manually increase or decrease the number of replicas (using kubectl).

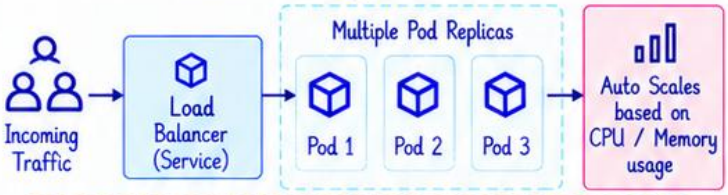
Horizontal Scaling (More Pods)

Increases the number of pod replicas (Handles more traffic).

Vertical Scaling (More Resources)

Increases CPU/Memory for each pod (Less common, limited by node capacity).

How Horizontal Scaling Works ?



More traffic = more pods. Less traffic = fewer pods.

Manual Scaling Example

```
# Scale a deployment to 5 replicas
kubectl scale deployment my-app --replicas=5

# Check the status
kubectl get pods
kubectl get deployment my-app
```

Quick way to handle sudden traffic spikes !

Scaling in Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3     # Initial number of replicas
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
       - name: app
         image: nginx:latest
```

replicas defines the desired number of pods.

What is HPA (Horizontal Pod Autoscaler) ?

HPA automatically adjusts the number of pod replicas based on metrics like CPU utilization, memory usage, or custom metrics.

HPA monitors the load and scales your application automatically – no manual intervention needed !

How HPA Works ?



If CPU > target → scale up | If CPU < target → scale down

HPA Example (YAML)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2              # Minimum number of pods
  maxReplicas: 10             # Maximum number of pods
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70   # Target CPU usage (70%)
```

Automatically scales pods based on CPU usage !

Check HPA Status

```
kubectl get hpa
kubectl describe hpa my-app-hpa
kubectl get pods -w
```

Monitor how HPA is scaling your pods in real-time !

Common Metrics for Autoscaling

Metric Type	Example	Use Case
CPU Utilization	averageUtilization: 70	Scale based on CPU usage
Memory Utilization	averageUtilization: 80	Scale based on memory usage
Custom Metrics	Requests per second	Scale based on business metrics (e.g., QPS, queue length)

Key Takeaways

- ✓ Use horizontal scaling to handle variable traffic
- ✓ HPA automatically scales based on metrics
- ✓ Set appropriate min and max replicas
- ✓ Monitor scaling behavior with kubectl and metrics
- ✓ Helps in building reliable and cost-efficient applications

Handle Growth Keep Performing Stay Reliable !

@codedsoul_05

Scale Today For a Bigger Tomorrow !

From Concepts to Real Systems !

12. StatefulSets + DaemonSets + Jobs + CronJobs

Right Workload Right Controller !

@codedsoul_05

StatefulSets

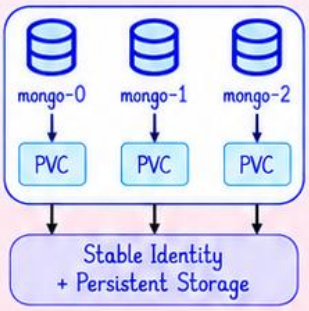
For stateful applications that need stable identity and storage.

- Provides stable, unique network identity (pod-0, pod-1, ...)
- Maintains order during scaling, updates, and deletion
- Each pod has its own persistent storage (PVC)
- Commonly used for databases (e.g., Kafka, MongoDB, Elasticsearch)

Use StatefulSet when pod identity and data persistence are important.

Example (StatefulSet YAML)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
spec:
  serviceName: "mongo"
  replicas: 3
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongo
        image: mongo:latest
        volumeMounts:
        - name: data
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
```



Each pod gets its own storage !

DaemonSets

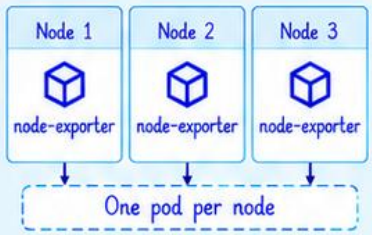
Runs a copy of a pod on every (or selected) node.

- Ensures one pod runs on each node (or a subset of nodes)
- Useful for cluster-level services (logging, monitoring, networking)
- Automatically handles new nodes (pods are scheduled there too)
- Commonly used for tools like Fluentd, Prometheus Node Exporter, Calico

Use DaemonSet for node-level services that must run everywhere.

Example (DaemonSet YAML)

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
      - name: node-exporter
        image: prom/node-exporter:latest
        ports:
        - containerPort: 9100
```



Runs on every node !

Jobs

Runs a task to completion (one-time job).

- Creates one or more pods that run until completion
- Useful for batch processing, data migration, backups, etc.
- Pods do not restart once the job is completed (unless specified)
- You can set parallelism and completions for multiple pods

Use Job for finite tasks that need to run and complete.

Example (Job YAML)

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: data-backup
spec:
  completions: 1      # Total successful runs
  parallelism: 1      # Run 1 pod at a time
  template:
    spec:
      containers:
      - name: backup
        image: alpine:latest
        command: ["sh", "-c", "echo Backup Done"]
      restartPolicy: Never
```

Runs and exits !



CronJobs

Runs tasks on a schedule (like a cron job).

- Creates Jobs on a defined schedule (cron syntax)
- Useful for periodic tasks (backups, reports, cleanup, etc.)
- Each schedule creates a new Job
- Supports concurrency policies (Allow, Forbid, Replace)

Use CronJob for recurring tasks.

Example (CronJob YAML)

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-job
spec:
  schedule: "0 2 * * *"     # Every day at 2 AM
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: cleanup
            image: alpine:latest
            command: ["sh", "-c", "echo Cleanup Done"]
          restartPolicy: OnFailure
  concurrencyPolicy: Forbid
```

Automate your tasks !

Runs on Schedule

Key Takeaways

- ✓ Use StatefulSet for stateful apps with stable identity and storage
- ✓ Use DaemonSet for node-level services
- ✓ Use Job for one-time tasks
- ✓ Use CronJob for scheduled, recurring tasks
- ✓ Choose the right controller based on your application's behavior and requirements

Automate Scale Operate With Kubernetes !

@codedsoul_05

Right Workload Happier Clusters Brighter Future !

From Concepts to Real Systems !

13. Security + RBAC + Service Accounts

@codedsoul_05

Secure Clusters Stronger Applications !

Why Security in Kubernetes ?

Kubernetes runs your applications, data and infrastructure. Security ensures only authorized users and workloads can access resources, prevents attacks, and keeps your cluster and applications safe.

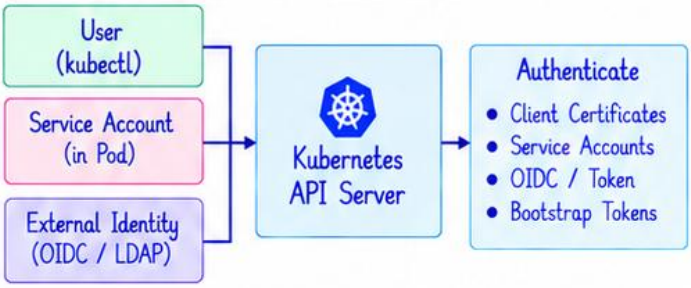
Security is not a feature, it's a necessity !

What is RBAC ?

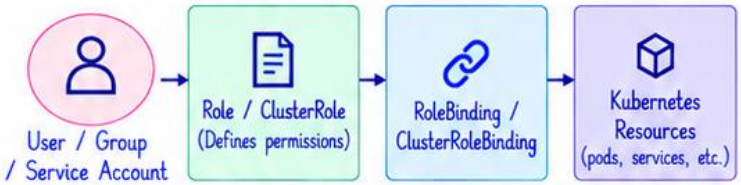
Role-Based Access Control (RBAC) controls who can do what on which resources in the cluster. You define Roles/ClusterRoles and bind them to users, groups or service accounts.

Principle of least privilege
Give only the access that's needed.

Authentication in Kubernetes



Authorization with RBAC



RBAC uses verbs (get, list, create, etc.) on resources (pods, services, etc.) within a namespace or cluster.

Service Accounts

A Service Account provides an identity for processes running in a pod. It is used by applications to securely access the Kubernetes API.

Each pod gets a service account, and a token is automatically mounted to access the API server.

Service Account Example (YAML)

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: default
```

Create a service account for your app !

RBAC Example (YAML)

1. Role (permissions within a namespace)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

2. RoleBinding (bind to user/SA)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-app-sa
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Common RBAC Verbs

Verb	Meaning
get	Read a single resource
list	List multiple resources
watch	Watch for changes
create	Create a new resource
update	Update an existing resource
patch	Partially update a resource
delete	Delete a resource

Role vs ClusterRole

Feature	Role	ClusterRole
Scope	Namespace	Cluster-wide
Use Case	Access to resources in a single namespace	Access to cluster-level resources (e.g., nodes)
Example Resources	pods, services, configmaps	nodes, persistentvolumes, clusterroles
Best Practice	Use Role (least privilege)	Use ClusterRole only when needed

Service Account in a Pod (YAML)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  serviceAccountName: my-app-sa
  containers:
  - name: app
    image: nginx
```

Pod uses the service account !

Key Takeaways

- ✓ Security is critical to protect your cluster and applications
- ✓ Use RBAC to control access with least privilege
- ✓ Service Accounts provide identity for applications running in pods
- ✓ Use Roles for namespace-scoped access and ClusterRoles for cluster-wide access
- ✓ Always follow the principle of least privilege

Secure Access. Trusted Workloads. Stronger Systems.

Learn Secure Build Better !

@codedsoul_05

Security Today A Safer Tomorrow !

From Concepts to Real Systems !

14. Monitoring + Logging + Troubleshooting

@codedsoul_05

Observe
Debug
Fix
Keep Going !

Why Observability is Important ?

Monitoring, logging and troubleshooting help you keep your Kubernetes applications healthy, reliable and performant. They allow you to detect issues early, understand what happened and fix problems quickly.

You can't improve what you can't see !

Key Components

Monitoring (Metrics)

Collects metrics like CPU, memory, pod status and helps in alerting.

Logging (Logs)

Collects logs from containers and system components for debugging and analysis.

Troubleshooting (Investigate & Fix)

Use kubectl and tools to identify, diagnose and resolve issues in the cluster.

Monitoring in Kubernetes

- Metrics are collected from nodes, pods and Kubernetes components.
- Helps in tracking resource usage, performance and availability.
- Commonly used tools: Prometheus (metrics collection), Grafana (visualization), Kubernetes Metrics Server.

Kubernetes Components
- Nodes
- Pods
- Services
- etc.

→ Prometheus (Collects Metrics) → Grafana (Visualize & Alert) → Dashboards & Alerts

Monitor today, avoid outages tomorrow !

Logging in Kubernetes

- Logs help you understand what is happening inside your containers and cluster components.
- Common tools: EFK (Elasticsearch + Fluentd + Kibana), Loki (with Grafana), or managed solutions (e.g., Cloud Logging).

Container Logs → Fluentd (Collects & Ships) → Elasticsearch (Stores & Indexes) → Kibana (Search & Visualize)

Logs tell the story behind the metrics !

Useful Monitoring Commands

```
# Check node and pod status
kubectl get nodes
kubectl get pods -A

# Check resource usage (requires metrics-server)
kubectl top nodes
kubectl top pods

# Describe a pod (for detailed info)
kubectl describe pod <pod-name>
```

Quickly check what's running and how it's performing !

Useful Logging Commands

```
# View logs of a pod
kubectl logs <pod-name>

# Follow logs in real-time
kubectl logs -f <pod-name>

# Logs from a specific container
kubectl logs <pod-name> -c <container-name>

# Previous container logs (if restarted)
kubectl logs <pod-name> --previous
```

Logs help you find the why !

Common Troubleshooting Steps

1	Check the status	kubectl get pods, kubectl get nodes
2	Describe the resource	kubectl describe pod <pod-name>
3	Check logs	kubectl logs <pod-name>
4	Investigate events	kubectl get events --sort-by=.metadata.creationTimestamp
5	Check resource usage	kubectl top pods / kubectl top nodes
6	Debug inside the pod	kubectl exec -it <pod-name> -- /bin/sh
7	Check configuration	kubectl get configmap, kubectl get secret
8	Look at network issues	kubectl get svc, kubectl get endpoints

Common Issues and How to Fix

Issue	Possible Cause	Fix
Pod not starting	Image pull error, config issue	Check logs and events, verify image and configs
CrashLoopBackOff	Application error, missing env vars	Check logs, fix app config, verify environment variables
High CPU / Memory	Resource limits too low	Check with kubectl top, adjust requests and limits
Pod not reachable	Service / network issue	Check service, endpoints, network policies
Logs not visible	Logging agent not running	Verify Fluentd/Loki, check pod logs
Node not ready	Resource pressure, network issue	Check node status and events, resolve underlying issue

Key Takeaways

- ✓ Use monitoring to track health and performance
- ✓ Centralize logs for easy debugging and analysis
- ✓ Follow a structured approach for troubleshooting
- ✓ Set up alerts for critical issues
- ✓ Observability leads to more reliable and resilient applications

Clear Logs
Calm Mind
Stable Clusters !

Monitor
Debug
Solve
Build Better !

Problems Will Happen
Be Ready !

@codedsoul_05

From Concepts to Real Systems !

15. Kubernetes Cheat Sheet + Top 100 Interview Questions

Practice Learn Crack Your Next Interview !

@codedsoul_05

kubectl Cheat Sheet (Most Used Commands)

Your daily Kubernetes toolkit !

Check cluster info	`kubectl cluster-info`
Get all resources	`kubectl get all`
Get pods	`kubectl get pods`
Get pods (with details)	`kubectl get pods -o wide`
Describe a resource	`kubectl describe pod <pod-name>`
View logs	`kubectl logs <pod-name>`
Follow logs (real-time)	`kubectl logs -f <pod-name>`
Execute inside a pod	`kubectl exec -it <pod-name> -- /bin/sh`
Apply a configuration	`kubectl apply -f <file.yaml>`
Delete a resource	`kubectl delete -f <file.yaml>`
Get services	`kubectl get svc`
Get deployments	`kubectl get deployments`
Get nodes	`kubectl get nodes`
Check events	`kubectl get events --sort-by=.metadata.creationTimestamp`
Check resource usage	`kubectl top pods`
Scale a deployment	`kubectl scale deployment <name> --replicas=3`
Restart a deployment	`kubectl rollout restart deployment <name>`

Useful One-Liners

Small commands Big impact !

```
kubectl get pods --all-namespaces          # Get all pods in all namespaces
kubectl get pods -o wide                   # Get pod details with node info
kubectl get events --sort-by=.metadata.creationTimestamp  # Check recent events
kubectl top nodes                          # Check node resource usage
kubectl top pods                           # Check pod resource usage
kubectl top pvc                            # Get persistent volumes claims
kubectl get pv                             # Get persistent volumes
kubectl get ingress                        # Get ingress resources
kubectl get hpa                            # Get horizontal pod autoscalers
kubectl get sa                             # Get service accounts
kubectl get roles,rolebindings             # Get RBAC resources
kubectl get cm,secret                      # Get ConfigMaps and Secrets
```

Resource Types Quick Reference

Resource	Purpose	Key Field
Pod	Runs containers	spec.containers
Deployment	Manages stateless apps	spec.replicas
StatefulSet	Manages stateful apps	spec.serviceName
DaemonSet	Runs on every node	spec.template
Job	Runs a task to completion	spec.completions
CronJob	Runs scheduled tasks	spec.schedule
Service	Exposes applications	spec.type
Ingress	Manages external access	spec.rules
ConfigMap	Stores non-sensitive config	data
Secret	Stores sensitive data	data
PersistentVolume	Storage in cluster	spec.capacity
PersistentVolumeClaim	Requests storage	spec.resources.requests
ServiceAccount	Identity for pods	automountServiceAccountToken
Role/ClusterRole	Defines permissions	rules
RoleBinding/ClusterRoleBinding	Binds permissions	subjects

Know your resources inside out !

Common YAML Templates

Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: app
    image: nginx:latest
```

Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 80
```

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: app
        image: nginx:latest
```

ConfigMap

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "prod"
  API_URL: "https://api.example.com"
```

Top 100 Kubernetes Interview Questions

1	Kubernetes Basics	• What is Kubernetes? • Why do we need Kubernetes? • Explain its architecture.	Q1 - Q10
2	Pods & Containers	• What is a Pod? • How are containers different from pods? • How do you share data between containers?	Q11 - Q20
3	Deployments & ReplicaSets	• What is a Deployment? • How does a ReplicaSet work? • How do you perform a rolling update?	Q21 - Q30
4	Volumes & Storage	• What are Persistent Volumes (PV)? • What is a Persistent Volume Claim (PVC)? • Difference between emptyDir and hostPath?	Q31 - Q40
5	Networking & Ingress	• What is a Service? • Explain Ingress and its use cases. • Difference between ClusterIP, NodePort, LoadBalancer?	Q41 - Q50
6	Scaling & HPA	• What is Horizontal Pod Autoscaler? • How does HPA work? • How do you scale applications in Kubernetes?	Q51 - Q60
7	StatefulSets, DaemonSets, Jobs, CronJobs	• When to use StatefulSet? • What is the use of a DaemonSet? • Difference between Job and CronJob?	Q61 - Q70
8	Security & RBAC	• What is RBAC? • What are Service Accounts? • How do you secure secrets in Kubernetes?	Q71 - Q80
9	Monitoring, Logging & Troubleshooting	• How do you monitor a Kubernetes cluster? • How do you view pod logs? • What steps do you take when a pod is not starting?	Q81 - Q90
10	Advanced & Real-World Scenarios	• How would you design a highly available app? • How do you handle failures and disaster recovery? • Scenario-based and tricky questions.	Q91 - Q100

Key Takeaways

- ✓ Practice hands-on with real clusters
- ✓ Understand concepts, not just commands
- ✓ Be ready for scenario-based questions
- ✓ Use this cheat sheet for quick revision
- ✓ Keep learning and building !

Learn Practice Build Get Hired !

Same Concepts Bigger Opportunities !

Kubernetes Today Better Systems Tomorrow !

@codedsoul_05